Kural: PRD and Backend API

Oct 26, 2026 · @Tharun B L

Kural turns a spoken complaint plus a photo into the right government document, privately, for a 12-hour Gemma 4 hackathon build.

Overview

The goal is a working demo where a Tamil voice complaint plus a photo becomes a ready-to-file document in under 15 seconds.

Problem. People face civic problems daily (broken roads, dead streetlights, garbage) but don't know which department to contact, which complaint type to file, or how to write it. Wrong-format complaints get ignored. Existing AI tools only draft RTI applications, and send data to the cloud.

Users. Residents who prefer Tamil, including elderly and less tech-confident people, helped by an operator at a kiosk such as an e-Sevai centre. Students who want a quick, correct complaint.

Goals

- Pick the right route (local body, CM Helpline or RTI) and department using fixed rules, never the model's guess.
- Produce a bilingual Tamil and English document with photo evidence and portal-ready fields.
- Explain next steps in Tamil and English through an offline Tamil text-to-speech engine.
- Run offline on a laptop kiosk so complaints never leave the machine. A phone app on Gemma 4 E2B is the roadmap, not this build.

Demo success metrics

Metric	Target
Correct route on our 15 test cases	13 of 15 or better
Voice + photo to draft (online)	Under 15 seconds
Same flow offline (E4B on Ollama)	Completes without errors
Legal wording or department invented by the model	Never

Scope

Ten P0 features must work for the demo; four P1 features are built only if P0 is done by hour 6.

#	Feature	Priority	Done when
1	Voice input	P0	Tamil or English recording with a 30-second countdown; longer audio split into 30-second chunks; typed input as fallback
2	Photo evidence and claim check	P0	Gemma describes 1–3 photos and flags a mismatch with the spoken claim (e.g. "pothole" but photo shows garbage) before drafting
3	Problem understanding	P0	Gemma fills structured fields only; it never chooses the route
4	Follow-up questions	P0	Missing required fields are asked in Tamil, never invented
5	Rule-based routing	P0	A rule table picks the route and shows the rule id that fired
6	Department lookup	P0	Department comes from our JSON table
7	Complaint history	P0	Every filed complaint is saved in SQLite; past complaints feed the routing rules
8	Editable preview	P0	Only Gemma-written text and personal details are editable
9	Bilingual PDF and portal fields	P0	Tamil and English PDF with photos, plus each portal form field with a copy button
10	Spoken next steps	P0	Next steps read aloud by an offline Tamil TTS engine
11	GPS ward detection	P1	Browser location, then photo GPS, then spoken location, matched to one city's ward boundary file
12	Tracker screen	P1	Saved complaints with deadline countdown
13	One-tap escalation	P1	Moves a complaint one step up the routing ladder

#	Feature	Priority	Done when
14	WiFi-off demo	P1 (first)	Full flow on Gemma 4 E4B via Ollama with no network

Routing rules

Gemma extracts; these rules decide. Rules are checked top to bottom and the first match wins.

Rule	When	Route
R1	User wants records or information, not a fix	RTI
R2	Civic problem, no earlier complaint	Local body complaint (corporation or municipality)
R3	Earlier local body complaint ignored, or problem spans departments	CM Helpline grievance
R4	CM Helpline grievance ignored after escalation	RTI asking for action taken
R5	Not a civic problem	No route; tell the user Kural doesn't cover it

The CM Helpline has three escalation levels (designated officer, first appellate officer, second appellate officer), so one-tap escalation climbs those before R4 fires. RTI is always a separate path, never mixed into a grievance.

Out of scope: consumer complaints, login and accounts, submitting to government portals, WhatsApp bot, languages beyond Tamil and English, a phone app, checking officials' replies (future work).

Go/no-go checks

Three checks in hour 1 decide the speech stack; each has a fallback ready.

Check	Go	No-go fallback
Tamil speech to text on Gemma 4 E4B via Ollama	Accurate Tamil transcript from a 20-second clip	Offline Whisper or an AI4Bharat Tamil ASR model

Check	Go	No-go fallback
Tamil speech to text via Gemini API (online)	Accurate transcript	Browser Web Speech API with `ta-IN`
Offline Tamil text to speech	Clear, understandable Tamil sentence	Show next steps as text only

Also in hour 0: open the real CM Helpline form and record its exact field names, and find a ward boundary file for the demo city (none found = feature 11 stays out).

12-hour timeline

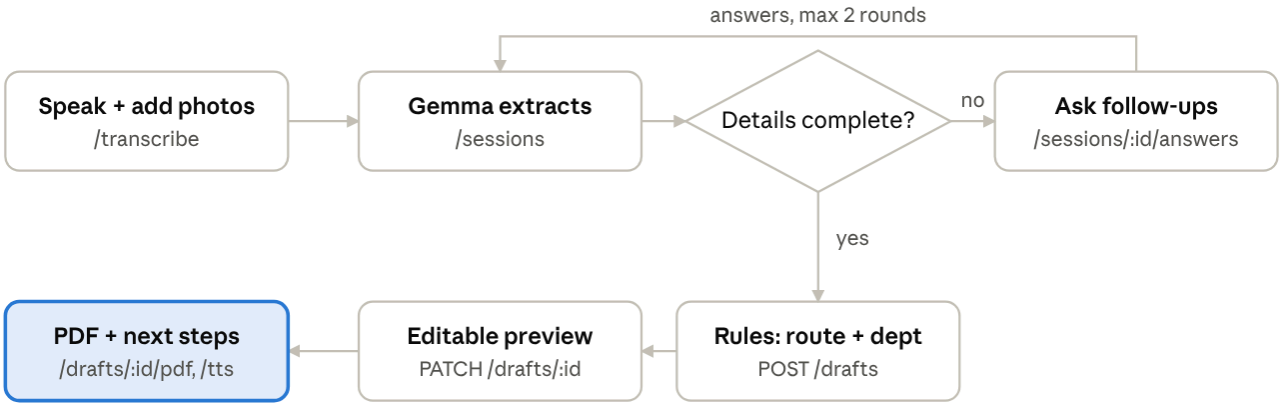
Feature freeze at hour 8; the last 4 hours are for bugs and two full rehearsals of the WiFi-off demo.

Hours	Front end	AI pipeline	Rules, templates, data
0–1	Repo, React shell, recorder with countdown	Speech go/no-go tests	CM Helpline field names, ward file, TTS test
1–4	Photo upload, follow-up question UI	Extraction prompt, claim check, schema validation	Routing rules, department JSON, 3 bilingual templates
4–6	Preview, copy buttons, PDF download	Ollama switch, chunked audio	SQLite history, 15 test cases
6–8	P1: tracker or GPS ward	P1: WiFi-off run on E4B	Run tests, fix wrong routes
8–12	Feature freeze: bugs, 2 full rehearsals, pitch		

User flow

A draft is only created once Gemma has every essential detail and the photo matches the claim; the routing rules, not Gemma, then pick the route and department.

Missing details loop back to Gemma before any draft is made



request flow · 7 steps, 1 decision · paths under /api

The department lookup, template and PDF steps never call the model, so they behave the same online and offline.

Non-functional requirements

Privacy and predictable output matter more than features.

- **Privacy:** in offline mode no request leaves the kiosk laptop. Photos, audio and personal details are stored only in local SQLite and the local uploads folder. No analytics or third-party logging.
- **Hardware:** Gemma 4 E4B runs on the kiosk laptop (16 GB RAM target); test speed in hour 0.
- **Audio:** Gemma accepts at most 30 seconds per clip at 16 kHz mono; the recorder shows a countdown and the backend splits longer audio.
- **Speed:** online flow under 15 seconds end to end; photos resized to max 1024 px before any model call.
- **Reliability:** every model response is validated against the Pydantic schema; one automatic retry on invalid output, then a clear error.
- **Correctness:** routes, departments, legal wording and deadlines come only from the rule table, JSON lookup and Jinja2 templates.
- **Tamil rendering:** PDFs use WeasyPrint with Noto Sans Tamil so Tamil text renders correctly.
- **Access:** served over HTTPS (Cloudflare Tunnel) so phone browsers allow microphone access.
- **Model switch:** one `MODEL_MODE` setting (`online` or `offline`) changes the backend; no other code path differs.

Data model

Seven Pydantic models carry the whole flow, from what Gemma extracts to the saved complaint.

```
from typing import Literal, Optional
from pydantic import BaseModel
from datetime import date, datetime

Route = Literal["local_body", "cm_helpline", "rti"]
Category = Literal["road", "streetlight", "garbage", "water", "drainage",
                   "other_civic", "not_civic"]

class Extraction(BaseModel):          # returned by Gemma, validated; no
route here
    category: Category
    summary_en: str                   # 2-3 sentence problem description
    summary_ta: str
    location_text: Optional[str]      # street / landmark as spoken
    duration: Optional[str]           # e.g. "3 months"
    severity: Literal["low", "medium", "high"]
    photo_findings: list[str] = []    # what the photos show
    photo_matches_claim: Optional[bool] = None   # None when no photo
    photo_mismatch_note: Optional[str] = None
    wants_records: bool = False       # asking for information, not a fix
(R1)
    user_says_ignored: bool = False   # "I complained before, nothing
happened"
    requested_info: list[str] = []    # RTI questions, if any
    missing_fields: list[str] = []    # e.g. ["location_text", "duration"]

class RouteDecision(BaseModel):       # produced by rules, not Gemma
    route: Route
    rule_id: Literal["R1", "R2", "R3", "R4"]
    reason_en: str
    reason_ta: str
    escalation_level: int = 1         # 1-3 within CM Helpline

class Department(BaseModel):          # from departments.json
    id: str
    name: str
    address: str
    submit_via: str                   # portal URL, email or office
    response_days: int

class Complainant(BaseModel):
```

```
    name: str
    address: str
    phone: Optional[str] = None

class Session(BaseModel):
    id: str
    transcript: str
    language: Literal["ta", "en"]
    photo_paths: list[str]
    lat: Optional[float] = None       # P1 ward detection
    lon: Optional[float] = None
    ward: Optional[str] = None
    extraction: Optional[Extraction]
    decision: Optional[RouteDecision]
    prior_complaint_ids: list[str] = []
    status: Literal["needs_info", "photo_mismatch", "not_civic", "ready"]

class Draft(BaseModel):
    id: str
    session_id: str
    decision: RouteDecision
    department: Department
    complainant: Complainant
    html: str                         # bilingual rendered template
    portal_fields: dict[str, str]     # CM Helpline field name -> value
    next_steps: dict[str, str]        # {"en": ..., "ta": ...}
    created_at: datetime

class Complaint(BaseModel):           # P0 history, stored in SQLite
    id: str
    draft_id: str
    route: Route
    category: Category
    location_text: Optional[str]
    ward: Optional[str]
    filed_on: date
    due_on: date                      # filed_on + department.response_days
    status: Literal["filed", "overdue", "resolved", "escalated"]
    parent_complaint_id: Optional[str] = None   # set when escalated from
another
```

Backend endpoints

Thirteen FastAPI endpoints under `/api`: ten for P0, three for P1.

Method	Path	Purpose	Feature	Priority
GET	`/api/health`	Status and current model mode	Setup	P0
POST	`/api/transcribe`	Audio to text, chunked at 30 s	1	P0
POST	`/api/sessions`	Extract fields, check photo, apply routing rules	2, 3, 4, 5	P0
POST	`/api/sessions/{id}/answers`	Answer follow-up questions or confirm a photo mismatch	2, 4	P0
GET	`/api/sessions/{id}`	Current session state	3	P0
POST	`/api/drafts`	Look up department, fill bilingual template, portal fields	6, 8, 9, 10	P0
PATCH	`/api/drafts/{id}`	Save user edits to the draft	8	P0
GET	`/api/drafts/{id}/pdf`	Download the bilingual PDF	9	P0
POST	`/api/tts`	Offline Tamil or English speech audio	10	P0
POST	`/api/complaints`	Save a filed complaint to history	7	P0
GET	`/api/complaints`	List with days left	12	P1
POST	`/api/complaints/{id}/escalate`	Move one step up the routing ladder	13	P1
PUT	`/api/config/mode`	Switch online / offline	14	P1

GET /api/health

```
{ "status": "ok", "model_mode": "online", "model": "gemma-4" }
```

POST /api/transcribe

Request: `multipart/form-data` with `audio` (webm or wav) and `language` (`ta` or `en`). The backend converts to 16 kHz mono and splits anything over 30 seconds into 30-second chunks, then joins the transcripts.

```
{ "transcript": "எங்கள் தெருவில் ...", "language": "ta", "engine": "gemma" }
```

`engine` is `gemma` or `whisper`. When the browser's Web Speech API is used, transcription happens in the browser and skips this endpoint.

POST /api/sessions

Request: `multipart/form-data` with `text` (transcript or typed), `language`, `photos` (0–3 images), and optional `lat` / `lon` from the browser's location request (P1). The backend also reads GPS from photo EXIF when present, and checks SQLite for earlier complaints with the same category and location before applying the routing rules.

```
{
  "session_id": "s_8f2a",
  "status": "needs_info",
  "extraction": {
    "category": "streetlight",
    "summary_en": "The streetlight on the resident's road has not worked for
3 months despite 2 complaints.",
    "summary_ta": "...",
    "location_text": null,
    "duration": "3 months",
    "severity": "medium",
    "photo_findings": ["Unlit streetlight pole", "Dark road at night"],
    "photo_matches_claim": true,
    "photo_mismatch_note": null,
    "wants_records": false,
    "user_says_ignored": true,
    "requested_info": [],
    "missing_fields": ["location_text"]
  },
  "decision": {
    "route": "cm_helpline",
    "rule_id": "R3",
    "reason_en": "Your earlier complaints to the local body were ignored, so
this goes to the CM Helpline.",
    "reason_ta": "...",
    "escalation_level": 1
  },
  "ward": null,
```

```
  "follow_up_questions": [
    { "field": "location_text", "question_en": "Which street is this on?",
"question_ta": "இது எந்த தெருவில் உள்ளது?" }
  ]
}
```

`status` values: `needs_info` (follow-up questions pending), `photo_mismatch` (user must confirm or retake the photo), `not_civic` (rule R5, no draft possible) or `ready`. `decision` is filled by the rules once the extraction is complete.

POST /api/sessions/{id}/answers

```
{ "answers": { "location": "Gandhi Street, near the bus stand", "area": "Ward
12" } }
```

Response: same shape as `POST /api/sessions`, re-validated. Max 2 rounds of questions; after that, remaining fields stay blank for the user to fill in the preview.

GET /api/sessions/{id}

Returns the `Session` object, so the front end can recover after a page refresh.

POST /api/drafts

Only allowed when the session `status` is `ready`.

```
{
  "session_id": "s_8f2a",
  "complainant": { "name": "R. Priya", "address": "14 Gandhi Street",
"phone": "98xxxxxx10" },
  "route_override": null
}
```

Response:

```
{
  "draft_id": "d_31c9",
  "decision": { "route": "cm_helpline", "rule_id": "R3", "escalation_level":
1 },
  "department": { "id": "muni-electrical", "name": "...", "address": "...",
"submit_via": "...", "response_days": 30 },
  "html": "<html>...bilingual rendered template...</html>",
  "portal_fields": {
    "Grievance details": "...",
    "Communication address": "14 Gandhi Street ..."
  },
  "next_steps": { "en": "Submit at ... You should get a reply within 30
days.", "ta": "..." }
}
```

`portal_fields` keys must match the real CM Helpline form labels recorded in hour 0.

`route_override` lets the user or operator pick a different route; the draft records both the rule's choice and the override.

PATCH /api/drafts/{id}

```
{ "fields": { "summary_en": "edited text" }, "complainant": { "phone": "..."
} }
```

Re-renders the template and returns the same shape as `POST /api/drafts`. Only Gemma-written fields and complainant details are editable; legal wording stays fixed.

GET /api/drafts/{id}/pdf

Returns `application/pdf` with the photos appended as an evidence page. Filename: `kural_<route>_<draft_id>.pdf`.

POST /api/tts

```
{ "text": "...", "language": "ta" }
```

Returns `audio/wav` from the offline Tamil TTS engine chosen in the hour-1 check. Gemma does not produce speech. If no engine passes the check, this endpoint returns `501` and the front end shows text only.

PUT /api/config/mode (P1)

```
{ "mode": "offline" }
```

Response: same as `/api/health`. Returns `503` if Ollama is not running.

POST /api/complaints (P0)

```
{ "draft_id": "d_31c9", "filed_on": "2026-10-26" }
```

Response: the `Complaint` object, with `due_on` set from the department's `response_days`.

GET /api/complaints (P1)

```
[ { "id": "c_7a1", "draft_id": "d_31c9", "title": "Streetlight, Gandhi Street", "filed_on": "2026-10-26", "due_on": "2026-11-25", "days_left": 30, "status": "filed" } ]
```

`status` turns `overdue` automatically when `days_left` is below 0.

POST /api/complaints/{id}/escalate (P1)

No body. Moves the complaint one step up the ladder: local body to CM Helpline, CM Helpline level 1 to 2 to 3, then an RTI asking for the action taken (R4). Response: same shape as `POST /api/drafts`, linked by `parent_complaint_id`; the original complaint's `status` becomes `escalated`.

Errors and configuration

Every error returns the same JSON shape, and every setting lives in `.env`.

```
{ "error": { "code": "MODEL_INVALID_OUTPUT", "message": "Could not understand the complaint. Please try again or type it." } }
```

HTTP	Code	When
400	`BAD_INPUT`	Missing text, more than 3 photos, unsupported file type
404	`NOT_FOUND`	Unknown session, draft or complaint id
409	`SESSION_NOT_READY`	Draft requested while follow-up questions are pending
422	`MODEL_INVALID_OUTPUT`	Gemma output failed validation twice

HTTP	Code	When
422	`NO_DEPARTMENT`	No lookup match for category and area; user picks manually
503	`MODEL_UNAVAILABLE`	Gemini API or Ollama unreachable
504	`MODEL_TIMEOUT`	Model call over 30 seconds

`.env` settings

```
MODEL_MODE=online            # online | offline
GEMINI_API_KEY=...           # never commit
GEMINI_MODEL=<gemma 4 model id from the Gemini API>
OLLAMA_URL=http://localhost:11434
OLLAMA_MODEL=<gemma 4 e4b tag from the Ollama library>
STT_ENGINE=gemma             # gemma | whisper | indic   (set by hour-1
check)
TTS_ENGINE=<offline Tamil TTS chosen in hour 1>   # none = text only
AUDIO_CHUNK_S=30
MODEL_TIMEOUT_S=30
MAX_PHOTOS=3
DATA_DIR=./data              # SQLite file, uploads, departments.json,
rules.json
WARD_BOUNDARIES=./data/wards.geojson   # P1, demo city only
```

Project layout

```
backend/
  main.py            # FastAPI app, routers
  routers/           # sessions, drafts, complaints, system
  services/
    llm.py           # one generate() switching Gemini API / Ollama
    stt.py, tts.py
    lookup.py        # departments.json
    render.py        # Jinja2 + WeasyPrint
  prompts/extract.txt
  templates/         # grievance.html, consumer.html, rti.html
  data/departments.json
```